

Module 2 Lab Session 1: Type Safety and Domain Models

Field	Value
Module	M2 TypeScript 6.0 Essentials
Exercises	1 (Strict Mode), 2 (Interfaces), 3 (Type Guards + parseStudent)
After this session you can show	A strict-mode TypeScript project with exported Student, Course, and EnrollmentRecord models, plus a type guard that safely narrows unknown API data

Welcome to the TMS Frontend Team

In M1 you built the TMS domain engine in C# Student, Course, EnrollmentRecord, Quiz, LabAssignment. The backend now produces JSON responses for every API endpoint. But JSON carries no type information. A response like `{ id: "STU-001", gpa: 3.8 }` could contain anything: the id might arrive as a number after an API migration, or gpa might come back as the string "three point eight" when someone changes the serialization settings.

TypeScript is the type layer between the .NET backend and the Angular frontend you will build in M8. It lets you describe the shape of data at compile time so that mismatches surface in your editor not on a live dashboard where Yared is trying to submit final grades.

The existing frontend code uses plain JavaScript with no type safety. Grades are sorted wrong because of string-vs-number confusion (`"85" > "9"` evaluates to false string comparison compares character codes). Enrollment status is tracked with boolean flags that allow 27 impossible states. Timestamps are unreliable because the code uses the legacy Date object.

Your task across these three sessions is to build the TypeScript models and utilities that will power the TMS frontend. You will organise them the same way professional Angular projects do one file per model so they are ready to move into the Angular project in M8.

Exercise 1: Enforcing the Rules

The situation: The existing TMS frontend was set up with `strict: false`. Because of this, a grade input from an HTML form was treated as a string, and `"85" > "9"` evaluated to false a student with 85% appeared below a student with 9% on the grade dashboard. Nobody caught the bug for an entire semester.

In M1, you enabled `<Nullable>enable</Nullable>` in the C# project to catch null-safety issues at compile time. This is the TypeScript equivalent `strict: true` forces the compiler to flag type errors before your code ever runs.

Decision rule (use this in real projects):

- Always enable `strict: true` in `tsconfig.json`. This is non-negotiable for production TypeScript.
- Enable `noUncheckedIndexedAccess` separately it catches a class of bugs that even `strict: true` misses (array access returning undefined).

Trade-off you should know: Strict mode produces more compiler errors on day one. That is the point. Each error is a bug you would have shipped silently in plain JavaScript. Fix the code, not the config.

Step 1 Create the Project

```
mkdir tms-client && cd tms-client
mkdir models
npm init -y
npm install typescript @js-temporal/polyfill @types/node --save
```

The `models/` folder is where every domain type lives one file per model. This is the same structure you will use in Angular (M8), where each model lives in `src/app/models/`. Starting with clean file organisation now means you will not have to refactor later.

Step 2 Initialize the TypeScript Compiler

```
npx tsc --init
```

Step 3 Configure Strict Mode

Open the generated `tsconfig.json` and set the following flags:

```
{
  "compilerOptions": {
    "target": "es2025",
    "module": "nodenext",
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUncheckedIndexedAccess": true
  }
}
```

Step 4 Verify

```
npx tsc --showConfig
```

What you should see: Your resolved configuration with all strict flags enabled.

Troubleshooting:

Problem	Likely cause	Fix
error TS2307: Cannot find module	module not set to nodenext or no .ts file exists	Set "module": "nodenext" and create an index.ts file (even empty)
error TS18003: No inputs were found	No TypeScript files in the project	Create an index.ts file so the compiler has something to process
tsc: command not found	TypeScript not installed locally	Run npx tsc instead of tsc to use the local install

Step 5 Try Transpiling to the ./dist directory and see what is generated

```
npx tsc --outDir dist
```

further reading <https://www.typescriptlang.org/tsconfig/>

Exercise 2: TMS Domain Models

The situation: The C# backend defines Student, Course, and EnrollmentRecord you built these in M1. The frontend needs matching TypeScript interfaces so the API responses have a known shape. If the backend returns { id: string, grade: number } and the frontend expects { id: number, grade: string }, the app silently breaks at runtime.

In C#, a class must explicitly declare class Quiz : IGradable this is nominal typing. TypeScript uses structural typing: if an object has the right shape (the right properties with the right types), it satisfies the interface automatically. The name does not matter the shape is the model.

Decision rule (use this in real projects):

- Use readonly on ID fields and any property that should not change after creation. In C#, readonly is enforced by the CLR at runtime. In TypeScript, it is compile-time only the JavaScript output has no concept of it. The value is in catching mistakes during development.
- Use ? for optional properties. gpa?: number means the property may be undefined. With strictNullChecks enabled, the compiler forces you to handle the missing case.

Trade-off you should know: TypeScript's readonly vanishes in the compiled JavaScript. If someone modifies the object using raw JavaScript, the protection is gone. This is expected TypeScript guarantees only apply within the TypeScript codebase.

Step 1 Create the Model Files

Each model gets its own file inside the `models/` folder. This is how professional Angular projects organise types one file per domain concept. Every interface is exported so other files can import it.

Create `models/student.model.ts`:

```
import { Temporal } from "@js-temporal/polyfill";

export interface Student {
  readonly id: string;
  name: string;
  enrollmentDate: Temporal.Instant;
  gpa?: number; // Optional undefined until the student receives a grade
}
```

Create `models/course.model.ts`:

```
import { Temporal } from "@js-temporal/polyfill";

export interface Course {
  readonly id: string;
  title: string;
  capacity: number;
  startDate?: Temporal.PlainDate;
}
```

Create `models/enrollment.model.ts`:

```
import { Temporal } from "@js-temporal/polyfill";

export interface EnrollmentRecord {
  readonly studentId: string;
  readonly courseCode: string;
  enrolledAt: Temporal.Instant;
}
```

Why export? Without it, the interface only exists inside the file that defines it. When you try to import `Student` from another file (which you will do in Exercise 3, and again in M8), TypeScript says “module has no exported member.” Every reusable type must be exported.

Step 2 Test the Constraints

Create a file called `index.ts` in the project root (not inside `models/`). This is where you will run test code throughout the module:

```
import { Temporal } from "@js-temporal/polyfill";
import { Student } from "../models/student.model";
```

```
const student: Student = {
  id: "STU-001",
  name: "Hana Tadesse",
  enrollmentDate: Temporal.Now.instant(),
};
```

```
// Try these what does the compiler say?
student.id = "STU-999";
console.log(student.gpa.toFixed(2));
console.log(student.gpa?.toFixed(2) ?? "Not yet graded");
```

Notice the `import { Student } from "../models/student.model"` this is how you will import models everywhere: in test files, in services, and in Angular components (M8).

What you should see: The readonly assignment error and the null-safety error both appear at compile time. The safe access line compiles and prints Not yet graded.

Troubleshooting:

Problem	Likely cause	Fix
No error on <code>student.id = "STU-999"</code>	<code>strict: true</code> not set in <code>tsconfig.json</code>	Check your config <code>readonly</code> enforcement requires strict mode
Temporal is not defined	Missing import	Add <code>import { Temporal } from "@js-temporal/polyfill";</code> at the top of your file
Autocomplete not working on <code>student.gpa</code>	IDE not recognizing the optional property	<code>gpa</code> is number undefined use <code>?.</code> to safely access it

Exercise 3: Safe API Parsing (Type Guards and Unknown)

The situation: The TMS API returns JSON. JSON has no type information `response.json()` gives you `unknown`. The legacy code cast everything to `any`, which allowed invalid data to silently corrupt the grade dashboard.

```
// Legacy dangerous
function processStudent(data: any) {
  console.log(`GPA: ${data.gpa.toFixed(2)}`); // Crashes if gpa is missing or not a number
}
```

In M1, you used guard clauses to validate preconditions (if (student is null) throw new `ArgumentNullException(...)`). TypeScript's type guards serve the same purpose they prove to the compiler that a value has the shape you expect before you use it.

Decision rule (use this in real projects):

- Use `unknown` for any data from outside your application (API responses, CSV imports, `localStorage`, URL parameters). Never use `any`.
- Write a type guard (`value is Student`) when you need the compiler to narrow the type after the check. Use `parseStudent` (throw on failure) when invalid data should stop execution.

Trade-off you should know: Type guards return boolean the caller decides what to do with invalid data. Parse functions throw they guarantee a valid result or nothing. Use guards when partial success is acceptable (filtering an array of mixed data). Use parse functions when invalid data is a programming error that should fail loudly.

Step 1 Understand the Problem

If data is the number 42, what happens when the code reads `data.gpa`? With `any`, the compiler says nothing. At runtime, `(42).gpa` returns `undefined`, and `.toFixed(2)` on `undefined` crashes.

Step 2 Write a Type Guard

The type guard and parse function belong in the same file as the model they validate. Open `models/student.model.ts` and add below the interface:

```
export function isStudent(value: unknown): value is Student {
  return (
    typeof value === "object" &&
    value !== null &&
    "id" in value &&
    "name" in value &&
    typeof (value as Record<string, unknown>).id === "string" &&
    typeof (value as Record<string, unknown>).name === "string"
  );
}
```

Now in `index.ts`, import and use it:

```
import { Student, isStudent } from "./models/student.model";

function processStudent(raw: unknown) {
  if (isStudent(raw)) {
    const gpaDisplay = raw.gpa?.toFixed(2) ?? "Not yet graded";
    console.log(`Student ${raw.name} GPA: ${gpaDisplay}`);
  } else {
```

```

        console.error("Invalid student data received");
    }
}

```

Step 3 Test It

```

processStudent({ id: "STU-001", name: "Hana", gpa: 3.7 });
// Prints: Student Hana GPA: 3.70

```

```

processStudent(42);
// Prints: Invalid student data received

```

What you should see: The first call prints the student data with a formatted GPA. The second call prints the error message no crash.

Troubleshooting:

Problem	Likely cause	Fix
isStudent always returns false	typeof value === "object" comes after the in checks	"id" in 42 throws a runtime error check typeof first
Autocomplete does not work inside the if block	Return type is boolean instead of value is Student	The return type must be a type predicate: value is Student
raw.name shows a type error inside the guard block	Guard function is not returning value is Student	Check the function signature the predicate is what triggers narrowing

Exercise 3 Part B: Throwing on Invalid Data

The isStudent guard returns false for bad data the caller decides what to do. Sometimes you need the function to either succeed with a valid Student or blow up with a clear reason so the caller does not have to check. This is the parseStudent pattern the TypeScript equivalent of M1's CapacityReachedException that tells you exactly what went wrong.

Add parseStudent to models/student.model.ts, below isStudent:

```

export function parseStudent(raw: unknown): Student {
    if (typeof raw !== "object" || raw === null) {
        throw new TypeError(
            `Expected an object, received ${raw === null ? "null" : typeof raw}`,
        );
    }

    const obj = raw as Record<string, unknown>;

    if (typeof obj.id !== "string") {

```

```

    throw new TypeError(
      `Expected id to be a string, received ${typeof obj.id}`,
    );
  }

  if (typeof obj.name !== "string") {
    throw new TypeError(
      `Expected name to be a string, received ${typeof obj.name}`,
    );
  }

  return {
    id: obj.id,
    name: obj.name,
    enrollmentDate: Temporal.Now.instant(),
  };
}

```

Test it in index.ts:

```

import { parseStudent } from "./models/student.model";

console.log(parseStudent({ id: "STU-001", name: "Hana" }));
// Prints a valid Student object

parseStudent({ id: 42, name: "Test" });
// Throws: TypeError: Expected id to be a string, received number

```

What you should see: The first call returns a Student object. The second call throws a TypeError because id is a number, not a string. The error message names the field and the actual type this makes debugging real API mismatches straightforward.

Troubleshooting:

Problem	Likely cause	Fix
Function returns undefined instead of throwing	You wrote return instead of throw new TypeError(...)	Replace return with throw for the error cases
Error message is not descriptive	Missing field name and actual typeof value	Include the field name and typeof in the message string
Confusion about parseStudent vs isStudent	Different use cases	Use isStudent when the caller handles both paths. Use parseStudent when invalid data should stop execution immediately

Session 1 What You Can Now Show

Before moving to Session 2, confirm all five:

- ☐ `npx tsc --showConfig` shows `strict: true` with all flags enabled
- ☐ Your `models/` folder contains three separate files: `student.model.ts`, `course.model.ts`, `enrollment.model.ts`
- ☐ `index.ts` successfully imports `Student` from `./models/student.model` the import compiles without errors
- ☐ `isStudent({ id: "STU-001", name: "Hana" })` returns `true` and narrows to `Student` inside an `if` block
- ☐ `parseStudent({ id: 42, name: "Test" })` throws a `TypeError` with a descriptive message naming the field and actual type

[further reading Typescript cheetsheets](#)